

PRÉPARÉ PAR
DJENEBA BA

PRÉSENTÉ LE
02/07/2026

RAPPORT PROJET JAVA FX

THE STORY APP



SOMMAIRE

- 1 Présentation du projet
- 2 Modèle métier
- 3 Persistance des données
- 4 Architecture
- 5 Principes SOLID
- 6 Fonctions principales
- 7 Statistiques
- 8 Tests
- 9 Conclusion et Perspectives d'évolution

PRÉSENTATION DU PROJET

StoryForge est l'application de bureau que nous développons en Java avec JavaFX. L'idée de départ était simple : offrir à un auteur un endroit clair pour préparer une œuvre de fiction avant de l'écrire. Plutôt que d'éparpiller ses notes, il peut rassembler dans un même outil ses histoires, les personnages qui les peuplent et les scènes qui composent le récit, le tout sauvegardé durablement dans une base de données MySQL.

L'application permet de gérer entièrement les histoires (les créer, les consulter, les modifier, les supprimer et les rechercher par titre depuis l'accueil), d'y rattacher des personnages avec un rôle et un genre, et de découper chaque histoire en scènes ordonnées dotées d'un lieu, d'un statut d'avancement et de mots-clés. Une scène peut réunir plusieurs personnages, et un même personnage peut revenir d'une scène à l'autre. Les saisies sont contrôlées pour éviter d'enregistrer une fiche incomplète, et toute l'interface s'appuie sur une feuille de style maison.

MODÈLE MÉTIER

Le cœur de l'application repose sur quatre concepts. Story représente une histoire : un titre, un auteur, un résumé, un genre, et la liste de ses personnages et de ses scènes. Personnage décrit un protagoniste rattaché à une histoire, avec son nom, son rôle (protagoniste, antagoniste, figurant...), son genre et deux descriptions, l'une physique et l'autre liée à sa personnalité. Scene correspond à une scène du récit : titre, ordre, lieu, résumé, mots-clés, ainsi que les personnages qui y apparaissent. Enfin, StatutScene est une énumération qui fixe l'avancement d'une scène (brouillon, en cours, terminé).

Ces classes restent volontairement simples : elles ne contiennent que des attributs et leurs accesseurs, plus une petite méthode de validation côté Story. Côté relations, une histoire possède plusieurs personnages et plusieurs scènes, tandis qu'une scène et un personnage sont liés par une relation plusieurs-à-plusieurs. Le diagramme ci-dessous reprend l'ensemble des classes, regroupées par couche.

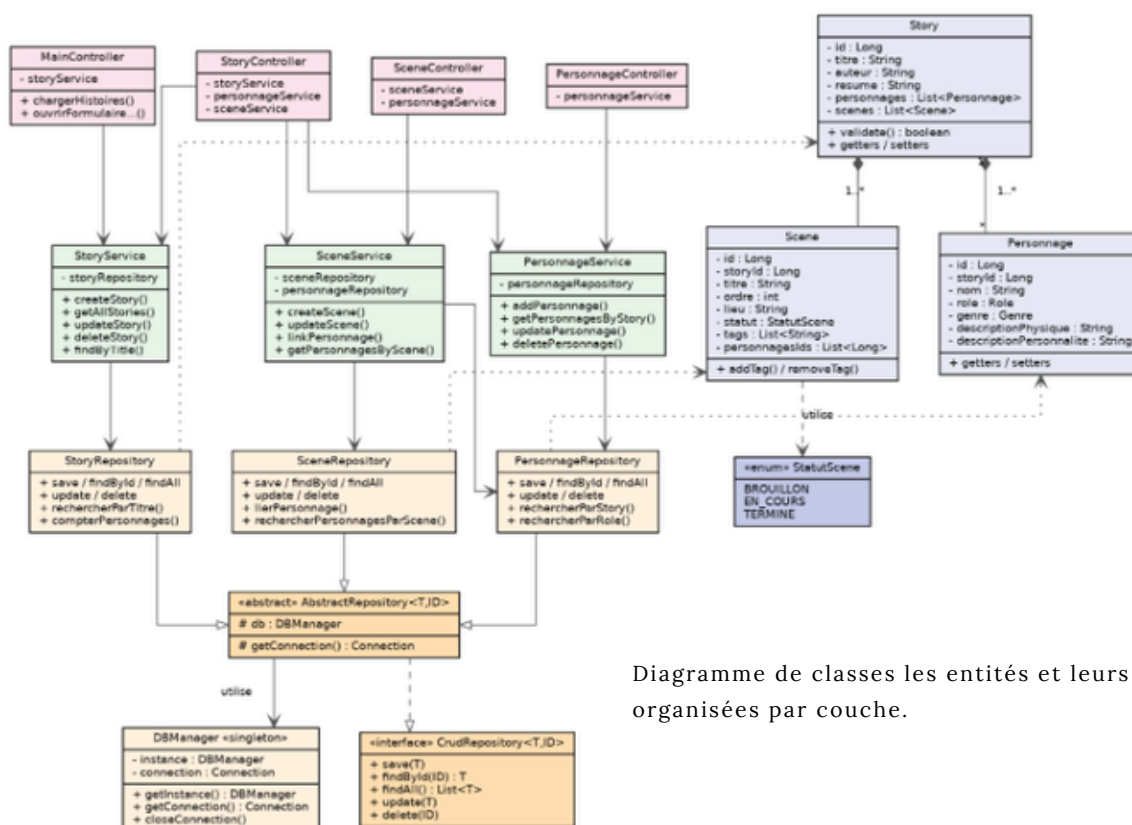
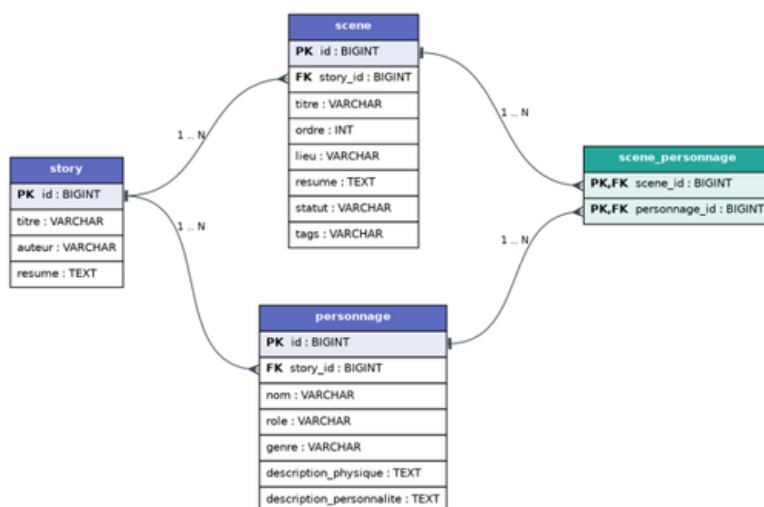


Diagramme de classes les entités et leurs relations, organisées par couche.

PERSISTANCE DES DONNÉES

Les données vivent dans une base MySQL nommée storyforge. La connexion passe par une seule classe, DBManager, construite en singleton : elle ouvre la connexion une fois et la partage avec tout le reste de l'application. Quatre tables structurent la base. story garde les histoires ; personnage garde les personnages, reliés à leur histoire par la colonne story_id ; scene garde les scènes, elles aussi reliées par story_id ; et scene_personnage est une table de liaison qui matérialise le lien plusieurs-à-plusieurs entre scènes et personnages.



Pour passer des objets Java aux lignes de la base, chaque entité a son propre dépôt (StoryRepository, PersonnageRepository, SceneRepository). À l'enregistrement, le dépôt exécute une requête INSERT via un PreparedStatement, récupère la clé générée par MySQL et la remplace dans l'objet. À la lecture, il reconstruit chaque objet à partir du ResultSet. Les requêtes sont paramétrées (ce qui écarte les injections SQL), les énumérations sont stockées sous forme de texte, et la liste de mots-clés d'une scène est enregistrée en une chaîne séparée par des virgules. Les captures suivantes montrent le contenu réel des tables après quelques saisies.

PERSISTANCE DES DONNÉES

id	titre	auteur	resume
1	Le retour du roi	Opowieść o	Nana, une femme pleine d'ambition, se retire...
2	Buh... Je sais pas	Fanta Be	C'est l'histoire d'une jeune femme qui veut tout...

Table story

id	story_id	nom	role	description_physique	genre	description_personnalite
1	1	Nana	ProtagonistePrincipale	Belle, noire, curvy mais taille fine,	Femme	Conquérante, pragmatique, manipulatrice
2	1	Rohan	AntagonisteSecondaire	Grand, boude, brun	Male	Imbue de sa personne mais au fond doux, an
3	1	Charles	Figurant	petit mais costaud	Male	Gentil mais niais

Table personnage

id	story_id	nom	role	description_physique	genre	description_personnalite
1	1	Nana	ProtagonistePrincipale	Belle, noire, curvy mais taille fine,	Femme	Conquérante, pragmatique, manipulatrice
2	1	Rohan	AntagonisteSecondaire	Grand, boude, brun	Male	Imbue de sa personne mais au fond doux, an
3	1	Charles	Figurant	petit mais costaud	Male	Gentil mais niais

Table de liaison scene_personnage : les couples (scene_id, personnage_id).

ARCHITECTURE

Nous avons organisé le projet en couches, chacune avec un rôle bien délimité, pour éviter de tout mélanger. La couche modèle ne fait que représenter les données (Story, Personnage, Scene). La couche persistance s'occupe uniquement de la base : DBManager pour la connexion, et les dépôts pour le SQL. Ces dépôts partagent une interface CrudRepository et une classe AbstractRepository qui factorise ce qu'ils ont en commun. La couche service (StoryService, PersonnageService, SceneService) porte les règles métier et coordonne les dépôts. Enfin, la couche contrôleur fait le pont entre l'écran et les services.

L'interface elle-même est décrite dans des fichiers FXML : main.fxml pour l'accueil et la liste des histoires, story-form.fxml pour le formulaire d'une histoire, personnage-form.fxml et scene-form.fxml pour les personnages et les scènes ainsi que dashboard.fxml pour les statistiques. Chaque FXML est piloté par son contrôleur (MainController, StoryController, etc.). Le programme démarre avec StoryForgeApp, lancé par la petite classe Launcher, et la fermeture de la fenêtre referme proprement la connexion à la base. En résumé, une action de l'utilisateur descend des écrans vers les contrôleurs, puis les services, puis les dépôts, et enfin la base et remonte dans l'autre sens.

```
StoryProject/
├── pom.xml
├── src/
│   ├── StoryProject/
│   │   ├── app/
│   │   │   ├── Launcher.java
│   │   │   └── StoryForgeApp.java
│   │   ├── controller/
│   │   │   ├── DashboardController.java
│   │   │   ├── MainController.java
│   │   │   ├── PersonnageController.java
│   │   │   ├── SceneController.java
│   │   │   └── StoryController.java
│   │   ├── model/
│   │   │   ├── Personnage.java
│   │   │   ├── Scene.java
│   │   │   ├── StatistiquesGlobales.java
│   │   │   ├── StatistiquesHistoire.java
│   │   │   ├── StatutScene.java
│   │   │   └── Story.java
│   │   ├── repository/
│   │   │   ├── AbstractRepository.java
│   │   │   ├── CrudRepository.java
│   │   │   ├── DBManager.java
│   │   │   ├── PersonnageRepository.java
│   │   │   ├── SceneRepository.java
│   │   │   └── StoryRepository.java
│   │   ├── service/
│   │   │   ├── AnalyseGlobaleService.java
│   │   │   ├── FiltreScenesService.java
│   │   │   ├── PersonnageService.java
│   │   │   ├── SceneService.java
│   │   │   ├── StatistiquesScenesService.java
│   │   │   └── StoryService.java
│   │   ├── view/
│   │   │   ├── dashboard.fxml
│   │   │   ├── main.fxml
│   │   │   ├── personnage-form.fxml
│   │   │   ├── scene-form.fxml
│   │   │   └── story-form.fxml
│   └── resources/
│       ├── css/
│       └── style.css
```

PRINCIPE SOLID

Notre découpage en couches applique d'abord le principe de responsabilité unique (SRP) : chaque classe n'a qu'un seul travail. DBManager gère la connexion et rien d'autre, les dépôts ne contiennent que du SQL, les services ne portent que la logique métier, et les contrôleurs ne s'occupent que de l'affichage. Concrètement, changer une requête SQL ne touche ni au métier ni à l'interface. Par exemple, StoryService vérifie qu'une histoire est valide avant de confier sa sauvegarde au dépôt, sans jamais écrire lui-même la moindre ligne de SQL :

```
public Story createStory(String titre, String auteur, String resume) {  
    Story story = new Story(titre, auteur, resume);  
    if (!story.validate())  
        throw new IllegalArgumentException("Titre et auteur obligatoires");  
    storyRepository.save(story);  
    return story;  
}
```

PRINCIPE SOLID

Nous appliquons aussi le principe ouvert/fermé (OCP). Grâce à l'interface CrudRepository et à AbstractRepository, on peut ajouter un nouveau dépôt (pour une future entité « Chapitre », par exemple) en héritant simplement de la classe abstraite, sans rien modifier de l'existant. Cela va de pair avec l'inversion des dépendances : les services ne créent pas leurs dépôts eux-mêmes, ils les reçoivent par leur constructeur, ce qui rend le code plus souple et plus facile à tester.

FONCTIONS PRINCIPALES

Fonctions principales

L'application StoryProject est un outil de gestion d'histoires pour écrivains, qui permet de structurer un univers narratif autour de trois entités liées entre elles : les histoires, les personnages et les scènes. Voici le détail des fonctionnalités, avec pour chacune la façon dont le code la porte.

Gestion des histoires Chaque histoire regroupe un titre, un auteur, un résumé et un genre littéraire (fantasy, science-fiction, romance, dystopie...). L'utilisateur peut créer, modifier, supprimer une histoire, et en rechercher une par son titre depuis la barre de recherche de l'écran d'accueil. Côté code, la classe Story porte ces données, StoryRepository s'occupe des requêtes SQL (INSERT, SELECT, UPDATE, DELETE), et StoryService applique les règles métier avant d'écrire en base, par exemple refuser d'enregistrer une histoire sans titre ni auteur via la méthode validate().

Gestion des personnages

Un personnage est rattaché à une histoire et décrit par un prénom, un nom, un âge, un rôle narratif (protagoniste principal, antagoniste secondaire, figurant...), un genre, une description physique et une description de la personnalité. La création, la modification et la suppression passent par PersonnageService, qui valide notamment que le nom n'est pas vide avant d'appeler le repository. C'est aussi ce service qui centralise la récupération de tous les personnages d'une histoire, utilisée à la fois dans l'écran de détail d'une histoire et dans le tableau de bord global.

Gestion des scènes

Une scène possède un titre, un ordre (pour respecter la chronologie du récit), un lieu, un résumé, un statut d'avancement (brouillon, en cours, terminé) et une liste de tags libres. Le statut est représenté par l'énumération StatutScene, ce qui garantit qu'on ne peut pas enregistrer une valeur incohérente en base. Les tags, eux, sont stockés sous forme de chaîne CSV dans une seule colonne, reconstituée en liste Java au chargement.

Association scènes et personnages

Une scène peut impliquer plusieurs personnages, et un personnage peut apparaître dans plusieurs scènes : c'est une relation plusieurs-à-plusieurs, portée en base par une table de liaison scene_personnage. Le code expose cette relation via les méthodes linkCharacter et unlinkCharacter de SceneService, appelées directement depuis le formulaire de scène, avec une vérification que le personnage existe bien avant de créer le lien.

FONCTIONS PRINCIPALES

Recherche et filtrage des scènes

Depuis l'écran d'une histoire, les scènes peuvent être filtrées par statut, par personnage impliqué, ou recherchées par mot-clé dans le titre, le résumé et les tags. Toute cette logique est isolée dans une classe dédiée, `SceneFilterService`, qui travaille uniquement sur les listes déjà chargées en mémoire avec des Streams Java, sans repasser par la base à chaque changement de filtre : c'est ce qui permet aux filtres de se ré-appliquer instantanément dès qu'un critère change.

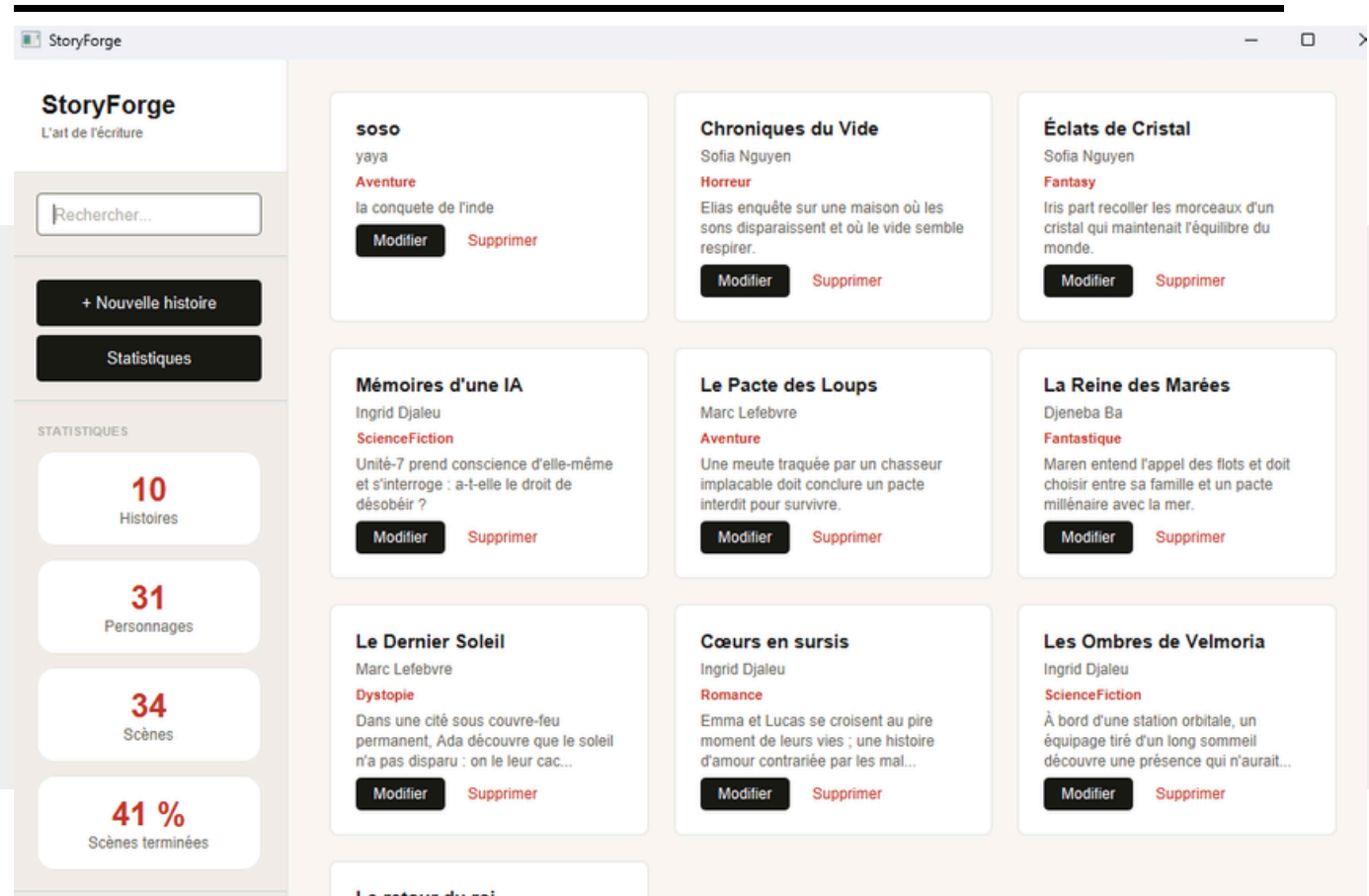
Tableau de bord statistique

Une page dédiée agrège les chiffres de tout l'atelier : nombre total d'histoires, de personnages, de scènes, pourcentage de scènes terminées, répartition des scènes par histoire, des personnages par rôle et des tags les plus utilisés. Ces calculs sont effectués par `GlobalStatsService`, qui parcourt l'ensemble des histoires et construit les statistiques à la volée, puis affichés sous forme de graphiques (barres et camembert) grâce aux composants `BarChart` et `PieChart` de JavaFX.

Architecture générale

L'ensemble du projet suit une architecture en couches strictes : les classes du dossier `model` ne contiennent que des données, les classes du dossier `repository` s'occupent exclusivement de l'accès à la base MySQL (via une interface commune `CrudRepository` et une classe `AbstractRepository` factorisant la connexion), les classes du dossier `service` portent la logique métier et les validations, et les classes du dossier `controller` font uniquement le lien entre l'interface graphique JavaFX (fichiers FXML) et les services. Cette séparation permet, par exemple, de modifier l'interface sans toucher à la logique de calcul des statistiques, ou de changer une requête SQL sans impacter l'écran qui l'utilise.

ÉCRAN D'ACCUEIL — LISTE DES HISTOIRES

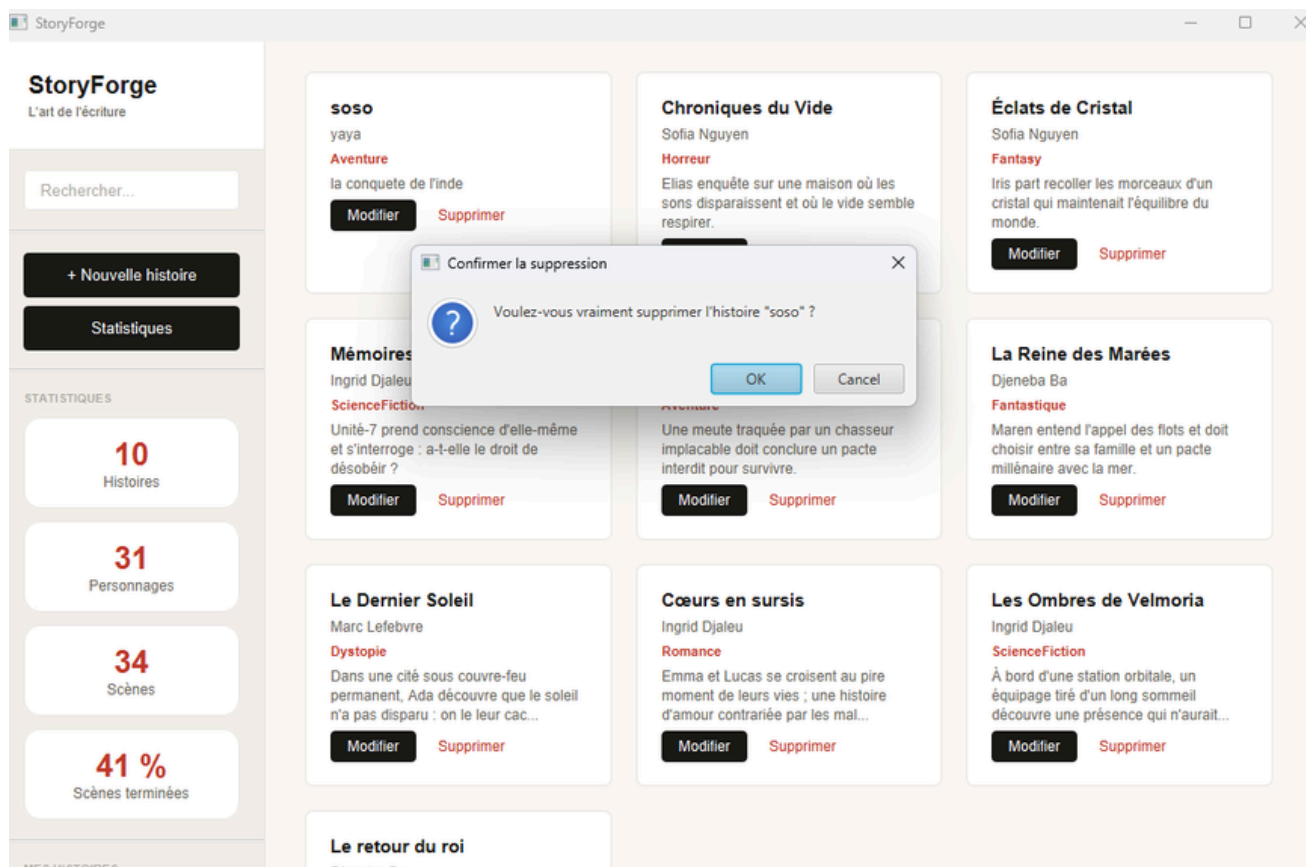


Élément	Comportement attendu	Vérification
Champ « Rechercher... »	Filter en direct la grille par titre au fur et à mesure de la saisie (touche relâchée). Champ vide = toutes les	Manuel (IHM)
Bouton « + Nouvelle histoire »	Ouvre le formulaire de création en fenêtre modale. Après sauvegarde, la grille est rechargée.	Manuel (IHM)
Bouton « Statistiques »	Ouvre le tableau de bord analytique (fenêtre séparée).	Manuel (IHM)
Carte « Histoires »	Affiche le nombre total d'histoires. Un clic déroule la liste des titres.	Unitaire
Carte « Personnages »	Affiche le nombre total de personnages de l'atelier (ou de l'histoire sélectionnée). Un clic déroule la liste.	Unitaire
Carte « Scènes »	Affiche le nombre total de scènes. Un clic déroule la liste.	Unitaire
Carte « Scènes terminées »	Affiche le pourcentage de scènes au statut Terminé sur le total des scènes.	Unitaire
Carte d'une histoire (titre, auteur, genre, résumé)	Un clic sur la carte (hors boutons) sélectionne l'histoire et met à jour les statistiques de gauche. La	Manuel (IHM)
Bouton « Modifier »	Ouvre le formulaire d'édition pré-rempli pour cette histoire.	Manuel (IHM)
Bouton « Supprimer »	Affiche une boîte de confirmation avant suppression réelle en base.	Intégration

FORMULAIRE « MODIFIER L'HISTOIRE »

Élément	Comportement attendu	Vérification
Lien « ← Accueil »	Revient à la grille des histoires sans sauvegarder.	Manuel (IHM)
Bouton « Supprimer » (en-tête)	Supprime l'histoire courante après confirmation, puis retour à l'accueil.	Intégration
Champ TITRE	Obligatoire. Un titre vide ou composé uniquement d'espaces doit être refusé à la sauvegarde.	Unitaire
Champ AUTEUR	Obligatoire, même règle que le titre.	Unitaire
Champ RÉSUMÉ + compteur de caractères	Facultatif. Le compteur se met à jour en direct (X / 500).	Manuel (IHM)
Bouton « + Ajouter » (Personnages)	Ouvre le formulaire de création de personnage lié à cette histoire.	Manuel (IHM)
Liste des personnages	Un personnage sans nom est refusé à la sauvegarde. Deux personnages de la même	Unitaire
Bouton « + Ajouter » (Scènes)	Ouvre le formulaire de création de scène liée à cette histoire.	Manuel (IHM)
Bloc STATISTIQUES (Personnages / Scènes /	Recalculé après chaque ajout, modification ou suppression de personnage ou de scène.	Unitaire
Champ « Rechercher un mot-clé »	Filtre les scènes affichées par titre, résumé ou tag.	Unitaire
Menu « Tous les statuts »	Filtre les scènes par statut (Brouillon / En cours / Terminé).	Unitaire
Menu « Tous les personnages »	Filtre les scènes où apparaît le personnage choisi.	Unitaire
Bouton « Réinitialiser »	Annule tous les filtres actifs et réaffiche toutes les scènes de l'histoire.	Manuel (IHM)
Boutons « Annuler » / « Sauvegarder »	Annuler ferme sans enregistrer ; Sauvegarder valide puis persiste les modifications en base.	Intégration

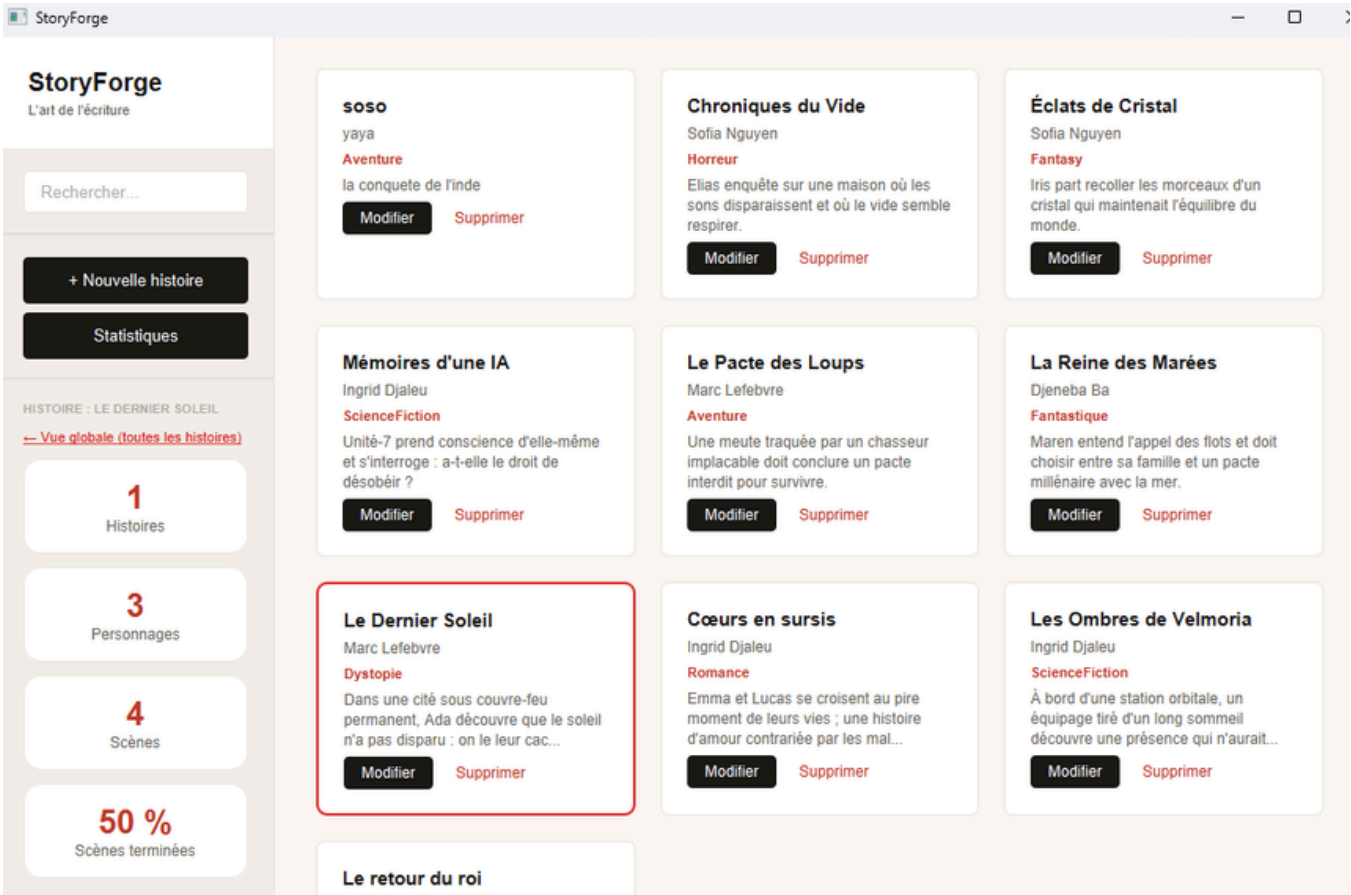
CONFIRMATION DE SUPPRESSION



Boîte de dialogue de sécurité affichée avant toute suppression définitive d'une histoire.

Élément	Comportement attendu	Vérification
Message de confirmation	Affiche dynamiquement le titre exact de l'histoire à supprimer.	Manuel (IHM)
Bouton « OK »	Confirme : l'histoire (et ses personnages/scènes liés) est supprimée en base, la grille se recharge.	Intégration
Bouton « Cancel »	Annule : ferme la boîte de dialogue sans aucune modification en base.	Manuel (IHM)

SÉLECTION D'UNE HISTOIRE DEPUIS L'ACCUEIL



Fonctionnalité permettant de consulter, sans quitter l'accueil, les statistiques d'une seule histoire en cliquant directement sur sa carte.

Élément	Comportement attendu	Vérification
Titre de section « HISTOIRE : ... »	Remplace le titre « STATISTIQUES » générique pour indiquer quelle histoire est affichée.	Manuel (IHM)
Lien « ← Vue globale »	Désélectionne l'histoire et restaure les statistiques globales de tout l'atelier.	Manuel (IHM)
Carte « Histoires »	Affiche 1 (une seule histoire est en contexte).	Unitaire
Carte « Personnages »	Affiche uniquement le nombre de personnages de l'histoire sélectionnée.	Unitaire
Carte « Scènes »	Affiche uniquement le nombre de scènes de l'histoire sélectionnée.	Unitaire
Carte « % Scènes terminées »	Recalculé sur la base des seules scènes de l'histoire sélectionnée.	Unitaire
Carte mise en surbrillance rouge	Indique visuellement quelle histoire est actuellement sélectionnée dans la grille.	Manuel (IHM)

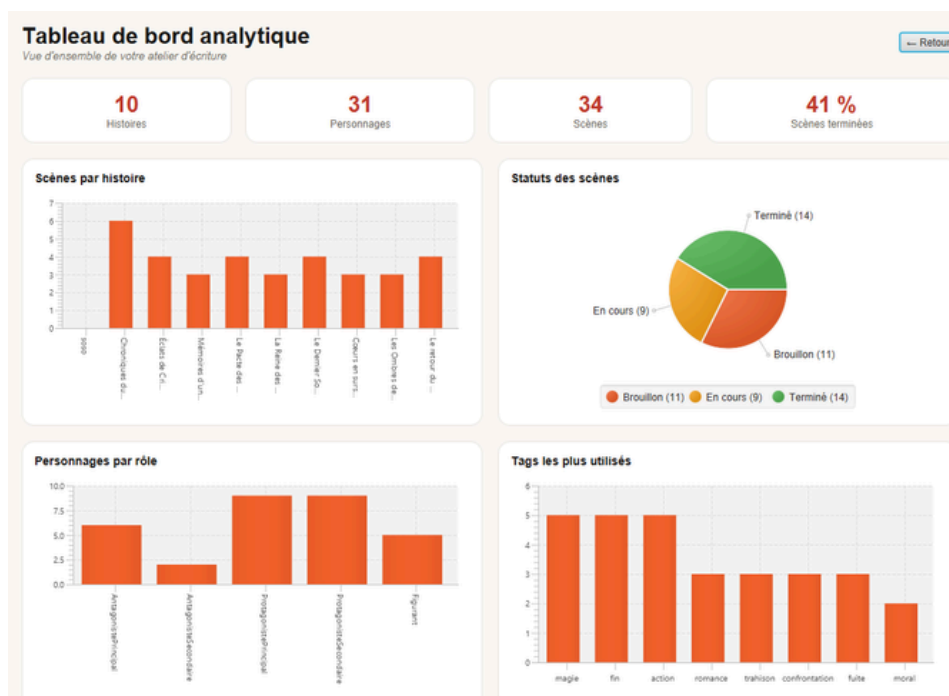
NOS STATISTIQUES

L'application propose un tableau de bord qui agrège des chiffres sur l'ensemble de l'atelier, toutes histoires confondues : nombre total d'histoires, de personnages, de scènes, pourcentage de scènes terminées, répartition des scènes par histoire, des scènes par statut, des personnages par rôle, et les tags les plus fréquemment utilisés.

Ce calcul est entièrement fait côté application, en Java, par une classe dédiée, GlobalStatsService. Elle ne fait pas de requête SQL d'agrégation directement en base : elle récupère les listes complètes d'histoires, de personnages et de scènes via les services existants (StoryService, PersonnageService, SceneService), puis calcule les statistiques en mémoire avec des Streams Java. Par exemple, la répartition des personnages par rôle se fait en regroupant la liste des personnages par leur champ role, et en comptant les occurrences de chaque valeur.

Le résultat de ce calcul est transporté par un objet StatistiquesGlobales, un record Java qui regroupe tous les chiffres calculés (nombre d'histoires, de personnages, de scènes, pourcentage de complétion, répartitions par histoire, par statut, par rôle et par tag). Ce choix d'un record plutôt que de plusieurs variables séparées permet de transmettre toutes les statistiques d'un seul bloc entre le service et le contrôleur, sans risque d'oublier un champ en cours de route.

Côté interface, c'est DashboardController qui appelle ce service au chargement de l'écran, puis répartit les résultats entre des indicateurs textuels (nombre d'histoires, de personnages, de scènes, taux de complétion) et des graphiques JavaFX : un BarChart pour les scènes par histoire, un PieChart pour la répartition par statut, et deux autres BarChart pour les personnages par rôle et les tags les plus utilisés. Une déclinaison de cette logique existe aussi à l'échelle d'une seule histoire, via SceneStatsService, qui calcule le nombre de personnages, le nombre de scènes et la répartition des scènes par statut pour l'histoire actuellement consultée, avec un résultat transporté cette fois par un record StatistiquesHistoire.



TESTS LOGICIELS

Les tests garantissent que les règles métier fonctionnent comme prévu et permettent de détecter les régressions lorsque le code évolue. On distingue deux familles de tests, complétées par la vérification manuelle de l'interface.

TESTS UNITAIRES ET FAKE REPOSITORY

Un test unitaire vérifie une classe ou une règle de manière isolée, sans base de données. Pour y parvenir, on substitue à l'accès MySQL un Fake Repository : une implémentation en mémoire (une simple liste). Cela est rendu possible par l'architecture en couches et l'inversion de dépendance : le service ne dépend que d'une abstraction de dépôt, que l'on peut remplacer en test. Chaque test suit la structure Arrange (préparer) → Act (exécuter) → Assert (vérifier), utilise les annotations `@Test` et `@BeforeEach` (environnement propre avant chaque test) et les assertions `assertEquals`, `assertTrue`, `assertThrows`, etc.

Classe de test	Ce qu'elle vérifie
StoryTest	Validation d'une histoire (titre, auteur, genre obligatoires)
StoryServiceTest	Création valide, refus des cas invalides, recherche, suppression
PersonnageServiceTest	Validation du nom, ajout, filtrage par histoire, suppression
SceneServiceTest	Titre/ordre, recherche, filtrage, liaison de personnages
StatistiquesScenesServiceTest	Comptages et répartition par statut
FiltreScenesServiceTest	Filtrage par statut / personnage / mot-clé et combinaison

TEST D'INTÉGRATION

Le test d'intégration vérifie la persistance réelle : enregistrer une histoire puis la relire depuis MySQL. Il dépend d'une base accessible et reste donc désactivé par défaut, afin que l'exécution des tests reste possible sans serveur de base de données.

```
[INFO] T E S T S
[INFO] -----
[INFO] Running StoryProject.model.StoryTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 s -- in StoryProject.model.StoryTest
[INFO] Running StoryProject.repository.SceneRepositoryIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.921 s -- in StoryProject.repository
.SceneRepositoryIntegrationTest
[INFO] Running StoryProject.service.PersonnageServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in StoryProject.service.PersonnageServiceTest
[INFO] Running StoryProject.service.SceneFilterServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in StoryProject.service
.SceneFilterServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.592 s
[INFO] Finished at: 2026-07-02T04:35:07+02:00
[INFO] -----
```

TRAÇABILITÉ : RÈGLES MÉTIER ET TESTS

Règle métier	Type de test	Classe de test
Une histoire ne peut pas avoir un titre vide	Unitaire	StoryServiceTest
Un personnage ne peut pas avoir un nom vide	Unitaire	PersonnageServiceTest
Une scène ne peut pas avoir un titre vide	Unitaire	SceneServiceTest
Une scène ne peut contenir un personnage hors de l'histoire	Unitaire	SceneServiceTest
Filtrage des scènes par statut / personnage	Unitaire	FiltreScenesServiceTest
Recherche des scènes par mot-clé	Unitaire	FiltreScenesServiceTest
Enregistrer une histoire puis la relire	Intégration	StoryRepositoryIntegrationTest

DIFFICULTÉS RENCONTRÉES

La première vraie difficulté a été la connexion à la base de données. Entre l'URL, les identifiants et une connexion qui se fermait, l'application refusait souvent de démarrer. Centraliser toute la connexion dans un DBManager unique, capable de la rouvrir au besoin, a réglé le problème une bonne fois pour toutes.

Les dépendances du pom.xml nous ont aussi pris du temps : tant que JavaFX et le pilote MySQL n'étaient pas déclarés avec les bonnes versions, rien ne se lançait. Il a fallu mettre soigneusement javafx-controls, javafx-fxml et mysql-connector-j, et configurer le plugin JavaFX pour pouvoir exécuter le projet. À cela s'est ajouté un conflit de nommage entre la classe javafx.scene.Scene et notre propre classe métier Scene, qui portent le même nom : dès qu'on utilisait les deux ensemble, il fallait préciser le nom complet pour lever l'ambiguïté.

Cette architecture posée trop tôt, avant que le besoin réel des écrans soit bien compris, a d'ailleurs montré ses limites une fois les formulaires en place. La table de liaison scène/personnage impose que la scène existe déjà en base avant qu'on puisse lui associer un personnage, alors que le formulaire de création permettait de sélectionner des personnages avant même le premier enregistrement. Il a fallu revoir le comportement du bouton de liaison pour qu'il déclenche une sauvegarde automatique de la scène dès le premier personnage ajouté. Cette correction a elle-même révélé un second bug, plus subtil : deux variables représentaient chacune, à leur manière, l'état "scène déjà enregistrée ou non", mais elles n'étaient pas mises à jour aux mêmes endroits.

Le bouton Sauvegarder se fiait à l'une, la liaison automatique à l'autre, ce qui provoquait la création de deux scènes distinctes pour une seule saisie. Ramener toute la logique sur une unique source de vérité a corrigé le problème durablement. Enfin, bien imaginer l'architecture et l'interface avant de coder a demandé beaucoup de réflexion. Découper proprement les couches et penser l'enchaînement des écrans n'est pas venu tout de suite : nous avons d'abord modélisé les classes et la base, puis maqueté les écrans, avant de nous lancer dans le code. D'autres points plus ponctuels nous ont occupés au passage, récupérer l'identifiant généré après un INSERT, représenter la relation plusieurs-à-plusieurs avec une table de liaison, ou encore stocker la liste de mots-clés d'une scène, mais ils se sont résolus une fois l'architecture posée.

Un dernier travail, plus transversal, a consisté à harmoniser la nomenclature de tout le code. Le projet a évolué progressivement d'une base entièrement en français vers une convention mixte, puis entièrement en anglais pour les noms de méthodes (modèle, repository, service, contrôleur), en conservant les noms de classes, les identifiants FXML et les variables locales en français. Ce renommage à grande échelle a demandé plusieurs passes pour rester cohérent d'un bout à l'autre du projet sans casser les liaisons entre le code Java et les fichiers FXML.

CONCLUSION

L'application ne gère actuellement ni authentification ni droits d'accès, ce qui limiterait son usage à un seul utilisateur à la fois sur une base partagée. On pourrait imaginer un back-end beaucoup plus pensé multi-utilisateur avec authentification et une mise en ligne avec Spring Boot.

StoryProject répond à l'objectif de départ : permettre à un écrivain de gérer une histoire, ses personnages et ses scènes, avec une architecture en couches claire et une persistance fiable en base MySQL.

Les principales difficultés (connexion à la base, relation scènes/personnages, cohérence de l'état lors de la sauvegarde) ont surtout appris à s'appuyer sur une seule source de vérité pour chaque donnée. L'ajout des tests a aussi révélé une limite d'architecture, l'absence d'interfaces entre services et repositories, qui reste la principale piste d'amélioration.

Le projet couvre aujourd'hui l'ensemble des fonctionnalités prévues : gestion complète des histoires, recherche, filtrage, statistiques, et un premier socle de tests automatisés.

